Full .NET Interview Handbook (C#, ASP.NET Core, EF Core, SQL, Patterns)

Style: short, interview-ready answers with cross-questions. Includes full programs for coding problems.

Section 1 – C# & OOP

Q1. What is the difference between Dispose() and Finalize() in C#?

Answer: Dispose() is an explicit cleanup method (IDisposable) called by the developer to release unmanaged resources deterministically. Finalize() (~destructor) is called by GC before reclaiming an object to give it a last chance to clean unmanaged resources; it is non-deterministic, has performance cost, and should be avoided unless wrapping unmanaged handles. Prefer using 'using' or try/finally with Dispose().

Example:

```
public class FileHolder : IDisposable
{
    private FileStream _fs;
    public FileHolder(string path) => _fs = File.OpenRead(path);

    public void Dispose()
    {
        _fs?.Dispose(); // deterministic
        GC.SuppressFinalize(this);
    }

    ~FileHolder() // Finalizer - avoid unless necessary
    {
        _fs?.Dispose();
    }
}

// Usage
using (var fh = new FileHolder("data.txt")) { /* work */ } // auto-dispose
```

Cross-Questions: When do you need a finalizer? How does SafeHandle simplify finalization? What's the cost of finalizers?

Q2. Explain out, ref, and in keywords in C#.

Answer: All pass by reference. 'ref' requires the variable to be initialized; callee can read/write. 'out' need not be initialized; callee must assign before return—good for returning multiple values. 'in' is readonly-by-ref; callee cannot modify and is useful for large structs to avoid copies.

Example:

```
static void Demo(ref int a, out int b, in int c)
{
    a += 5;       // ok
    b = 42;       // must assign
    // c = 5;     // ■ not allowed (readonly)
}
```

Cross-Questions: When would you choose 'out' vs returning a tuple? Can methods be overloaded by ref/out?

Q3. What are SOLID principles?

Answer: S: Single Responsibility—one reason to change. O: Open/Closed—open for extension, closed for modification. L: Liskov Substitution—subtypes can replace base types without breaking correctness. I: Interface Segregation—prefer focused interfaces. D: Dependency Inversion—depend on abstractions, not concretions.

Example:

```
// D + I (DI): depend on abstraction
public interface IPaymentGateway { void Pay(decimal amount); }
public class StripeGateway : IPaymentGateway { public void Pay(decimal a) { } }

public class CheckoutService
{
    private readonly IPaymentGateway _gw;
    public CheckoutService(IPaymentGateway gw) => _gw = gw;
    public void Checkout(decimal total) => _gw.Pay(total);
}
```

Cross-Questions: Give a violation of SRP; How does DI help with unit testing? Difference between DIP and DI container?

Q4. Difference between var, dynamic, and object?

Answer: var: compile-time inferred static type; still strongly typed. dynamic: bypasses compile-time checks—resolution happens at runtime; may throw if member missing. object: base type—requires casting/unboxing for specific members.

Example:

```
var x = 10;          // int
dynamic d = "hi";    // runtime binding
object o = 5;        // boxing int as object
```

Cross-Questions: Performance implications of dynamic? When does var fail (anonymous types across method boundaries)?

Q5. Difference between interface and abstract class?

Answer: Interface: contract only (can have default implementations since C# 8), no state; multiple interfaces allowed. Abstract class: can have state, fields, protected members, constructors; single inheritance.

Example:

```
public interface IFly { void Fly(); }
public abstract class Bird { public abstract void Eat(); }
public class Sparrow : Bird, IFly { public override void Eat() { } public void Fly() { } }
```

Cross-Questions: When to choose abstract class over interface? Can interfaces have fields?

Q6. What is the use of yield return in C#?

Answer: Creates an iterator state machine lazily producing elements; simplifies custom enumerations, supports deferred execution and streaming of large data.

Example:

```
static IEnumerable<int> Evens(int n)
```

```
{
    for (int i = 0; i <= n; i++) if (i % 2 == 0) yield return i;
}
```

Cross-Questions: How does 'yield' interact with try/finally? Difference from returning List?

Q7. Explain the use of async and await in C#.

Answer: They simplify asynchronous programming by composing tasks without blocking threads. 'async' marks a method that returns Task/Task/ValueTask; 'await' asynchronously waits for completion, freeing the thread.

Example:

```
public async Task<string> DownloadAsync(HttpClient http, string url)
{
    using var res = await http.GetAsync(url);
    return await res.Content.ReadAsStringAsync();
}
```

Cross-Questions: What's ConfigureAwait(false)? Difference between concurrency and parallelism?

Q8. What is dependency injection? Types?

Answer: DI is supplying an object's dependencies from the outside. Types: Constructor injection (most common), Property injection, Method injection. Benefits: loose coupling, testability, SRP.

Example:

```
// ASP.NET Core registration
builder.Services.AddScoped<IRepo, SqlRepo>();
```

Cross-Questions: AddScoped vs AddTransient vs AddSingleton? When would you use each?

Q9. What is middleware in .NET Core?

Answer: Middleware components are ordered delegates that handle HTTP requests/responses in a pipeline (logging, auth, routing). Order matters.

Example:

```
app.Use(async (ctx, next) => { /* pre */ await next(); /* post */ });
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Cross-Questions: Difference between middleware and filters? Where would you handle global exceptions?

Q10. Difference between Task, Thread, and async?

Answer: Thread: OS-managed execution unit. Task: a higher-level abstraction representing work (often scheduled on thread pool). async/await: language features to compose tasks without manual threading.

Cross-Questions: When to use Task.Run? Can async void be used?

Q11. What are generics and why better than non-generic collections?

Answer: Generics provide type safety, avoid boxing/unboxing, and improve performance and readability. Non-generic collections store object and need casting.

Example:

```
List<int> nums = new(); // vs ArrayList requiring casts
```

Cross-Questions: Covariance/contravariance in generics?

Q12. Explain boxing and unboxing in C#.

Answer: Boxing: value type -> object (heap). Unboxing: object -> value type. Frequent boxing hurts performance and GC.

Example:

```
object o = 5; // boxing
int x = (int)o; // unboxing
```

Cross-Questions: Where does boxing occur implicitly (string.Format, object params)?

Q13. Difference between IEnumerable, IQueryable, and List.

Answer: IEnumerable: in-memory forward-only iteration; operators run in memory. IQueryable: builds expression trees for remote queries (e.g., EF to SQL). List: concrete in-memory collection with indexing.

Cross-Questions: Why avoid calling .ToList() too early with EF?

Q14. What are extension methods?

Answer: Static methods that appear as instance methods on the extended type via 'this' modifier in the first parameter.

Example:

```
public static class StringExt { public static bool IsNullOrEmptyEx(this string s) => strir
```

Cross-Questions: How are conflicts resolved if instance method exists?

Q15. What is AsNoTracking() in EF Core?

Answer: Disables change tracking for queries, improving performance for read-only scenarios; returned entities are detached.

Cross-Questions: When must you avoid AsNoTracking (when updating entities)?

Q16. What is IHostedService?

Answer: An interface for background tasks in ASP.NET Core. Implement StartAsync/StopAsync; register via AddHostedService.

Example:

```
public class Worker : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct) { while (!ct.IsCancel
}
// builder.Services.AddHostedService<Worker>();
```

Cross-Questions: Difference between IHostedService and Windows Service?

Q17. Difference between .NET Core and .NET Framework?

Answer: .NET Framework: Windows-only, legacy, WebForms/WCF. .NET (Core): cross-platform, high-performance, modern tooling; unified runtime from .NET 5+.

Cross-Questions: Porting considerations from Framework to .NET?

Section 2 – ASP.NET Core

QA1. Use of ConfigureServices and Configure (old Startup) / minimal hosting equivalent?

Answer: ConfigureServices registers services in the DI container. Configure builds the middleware pipeline. In .NET 6+, these are merged into Program.cs via builder.Services and app.Use... etc.

Example:

```
// .NET 6+ Program.cs
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
var app = builder.Build();
app.UseRouting();
app.MapControllers();
app.Run();
```

Cross-Questions: Order of middleware? Where do you add Swagger/Auth/CORS?

QA2. What is Model Binding in ASP.NET Core?

Answer: It maps HTTP request data (route, query string, headers, body) to action method parameters and model properties.

Cross-Questions: Difference between [FromBody], [FromQuery], [FromRoute]?

QA3. How to secure Web APIs in ASP.NET Core?

Answer: Use HTTPS, authentication (JWT/bearer), authorization policies/roles, input validation, rate limiting, CORS, and secure headers.

Cross-Questions: How do you validate model inputs (FluentValidation/DataAnnotations)?

QA4. How do you implement JWT authentication?

Answer: Add JWT bearer authentication, validate issuer/audience/signing key; add [Authorize] on controllers.

Example:

```
// builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
//     .AddJwtBearer(options => options.TokenValidationParameters = new TokenValidationPar
```

Cross-Questions: JWT structure (header, payload, signature)? Token expiry/refresh?

QA5. What is middleware pipeline in ASP.NET Core?

Answer: Ordered sequence of middleware components handling requests. Each can short-circuit or call next.

Cross-Questions: Difference between Use, Run, and Map?

QA6. What are filters in ASP.NET MVC?

Answer: Attributes that run at different stages: Authorization, Resource, Action, Exception, Result. Useful for cross-cutting concerns.

Cross-Questions: Difference between middleware and filters responsibilities?

QA7. How can you handle exceptions globally in ASP.NET Core?

Answer: Use UseExceptionHandler/UseDeveloperExceptionPage, custom error handling middleware, ProblemDetails, and logging.

Cross-Questions: When would you use ExceptionFilter?

QA8. Attribute routing vs conventional routing.

Answer: Attribute routing uses route attributes on actions/controllers; conventional routing defines patterns globally. Attribute routing gives fine control, conventional is central and DRY for MVC.

Cross-Questions: How to create route constraints?

QA9. Create and consume a REST API in ASP.NET Core?

Answer: Create controllers, define DTOs, add routes; consume using HttpClient or third-party clients. Document with Swagger.

Cross-Questions: Idempotency of PUT vs POST?

QA10. What is Swagger and how do you implement it?

Answer: Swagger/OpenAPI documents APIs and offers UI. Add Swashbuckle, enable in Program.cs, use annotations for docs.

Cross-Questions: How to secure Swagger in production?

Section 3 – Entity Framework Core

QE1. Difference between Code First and Database First?

Answer: Code First starts from C# classes, creates/migrates schema. Database First scaffolds models from an existing DB.

Cross-Questions: How do migrations work?

QE2. Lazy Loading, Eager Loading, Explicit Loading?

Answer: Lazy: navigation properties loaded on access. Eager: Include() loads related data in query. Explicit: Load() called manually with context.Entry().

Cross-Questions: N+1 problem and how Include/Select projection helps?

QE3. Use of DbContext?

Answer: Represents session with DB; tracks changes; exposes DbSet; saves changes and manages queries.

Cross-Questions: How to manage unit of work and lifetime (Scoped)?

QE4. How do migrations work in EF Core?

Answer: Add-Migration creates a migration from model changes; Update-Database applies to DB. Uses a history table to track.

Cross-Questions: How to handle breaking schema changes in prod?

QE5. What is a shadow property in EF Core?

Answer: A property not defined in the CLR class but tracked by EF (e.g., CreatedAt). Access via ChangeTracker APIs.

Cross-Questions: When would you use shadow properties (FKs, auditing)?

Section 4 – LINQ

QL1. Difference between Select() and SelectMany()?

Answer: Select projects each element. SelectMany flattens sequences of sequences into a single sequence—used for one-to-many projections.

Example:

```
// SelectMany example
var allOrderLines = customers.SelectMany(c => c.Orders);
```

Cross-Questions: When does SelectMany change cardinality?

QL2. Explain GroupBy, Join, and Where in LINQ.

Answer: Where filters, GroupBy groups with keys, Join combines sequences on matching keys (inner join).

Example:

```
var q = from e in employees
        join d in depts on e.DeptId equals d.Id
        group e by d.Name into g
        select new { Dept = g.Key, Count = g.Count() };
```

Cross-Questions: Difference between query syntax and method syntax?

QL3. Difference between query syntax and method syntax in LINQ?

Answer: Both compile to the same method calls. Query syntax is SQL-like and readable for joins; method syntax is fluent and more expressive with lambdas.

Cross-Questions: When would you prefer method syntax?

Section 5 – SQL Server

QS1. Difference between DELETE, TRUNCATE, and DROP?

Answer: DELETE removes rows with logging and can have WHERE; TRUNCATE deallocates data pages (faster) and resets identity; DROP removes the object definition entirely.

Cross-Questions: Can TRUNCATE be rolled back inside a transaction?

QS2. What is a stored procedure? Why use it?

Answer: Precompiled SQL stored on server, improves reuse, performance (plan caching), security via permission abstraction.

Cross-Questions: Difference between stored proc and function?

QS3. What is a trigger? Types?

Answer: Trigger is a special proc that runs automatically on table/view events. Types: AFTER/INSTEAD OF, DML/DDL triggers.

Cross-Questions: Common use-cases and pitfalls (recursive triggers)?

QS4. What is an index? Types of indexes?

Answer: Index accelerates searches by maintaining ordered structures (B-tree). Types: clustered, nonclustered, composite, unique, full-text, columnstore.

Cross-Questions: Trade-offs: write overhead, storage.

QS5. Clustered vs non-clustered index?

Answer: Clustered defines the physical order of data (one per table). Nonclustered is a separate structure pointing to data (multiple allowed).

Cross-Questions: When to choose a composite key/index?

QS6. What is COALESCE() function?

Answer: Returns the first non-null expression among its arguments; ANSI-standard; similar to ISNULL but supports multiple args and proper type precedence.

Cross-Questions: COALESCE vs ISNULL differences?

QS7. Difference between ROW_NUMBER() and RANK()?

Answer: ROW_NUMBER gives unique sequence; RANK gives same rank for ties and leaves gaps.

Cross-Questions: When to use DENSE_RANK?

QS8. How can you optimize SQL queries?

Answer: Proper indexing, avoid SELECT *, reduce RBAR, sargable predicates, analyze execution plans, batch operations, cache results, use pagination, and correct data types.

Cross-Questions: What is a covering index? What is parameter sniffing?

QS9. How to find duplicate records in a table?

Answer: Group by keys and HAVING COUNT(*)>1; or use ROW_NUMBER partitioned by keys and delete where RN>1.

Example:

```
SELECT Col1, Col2, COUNT(*)
FROM T GROUP BY Col1, Col2 HAVING COUNT(*)>1;
```

Cross-Questions: How to delete duplicates while keeping the latest row?

QS10. Difference between primary key and unique key?

Answer: Both enforce uniqueness. PK also enforces NOT NULL and one per table (often clustered). Unique allows one NULL (SQL Server).

Cross-Questions: Composite primary keys—when and how?

Section 6 – Patterns & Architecture

QP1. What is the Singleton pattern and where do you use it?

Answer: Ensures a single instance and global access (e.g., cache, configuration). Thread-safe via static initialization.

Example:

```
public sealed class AppConfig
{
    private AppConfig() { }
    public static AppConfig Instance { get; } = new AppConfig();
}
```

Cross-Questions: Why is Singleton overused? How does DI scope compare?

QP2. What is the Repository pattern?

Answer: Abstracts data access behind a collection-like interface; decouples domain from persistence.

Cross-Questions: When is Repository + EF redundant?

QP3. What is Unit of Work?

Answer: Coordinates changes across repositories as a single transaction (DbContext is a UoW).

Cross-Questions: How to implement transactional boundaries in services?

QP4. Monolithic vs microservices architecture?

Answer: Monolith: single deployable unit—simple but less scalable. Microservices: independently deployable services—scalable but complex (observability, distributed tx).

Cross-Questions: Data consistency patterns: saga vs 2PC?

QP5. What is Clean Architecture?

Answer: Layered architecture with dependency rule: outer layers depend on inner (domain). Use cases independent of frameworks.

Cross-Questions: How do DTOs and mapping fit in?

Section 7 – Auth & API

QAAuth1. Token-based authentication & JWT?

Answer: Client receives a signed token after authentication; sends it in Authorization header. JWT has header, payload (claims), signature; server validates signature, issuer, audience, expiry.

Cross-Questions: How do you refresh tokens securely?

QAAuth2. Manage authenticated user across requests in ASP.NET Core?

Answer: Use authentication middleware and schemes; HttpContext.User carries ClaimsPrincipal; use cookies or bearer tokens.

Cross-Questions: Difference between authentication and authorization?

QAAuth3. FromBody vs FromHeader vs FromQuery in Web API?

Answer: FromBody: request body (JSON). FromHeader: HTTP headers. FromQuery: query string parameters. Model binding chooses defaults based on type.

Cross-Questions: When to use [FromRoute]?

Section 8 – Misc & Fundamentals

QM1. Value type vs reference type?

Answer: Value types live on the stack/by value semantics; reference types on the heap with references. Value types copy by default; ref types copy references.

Cross-Questions: Why are structs best kept small and immutable?

QM2. Memory leak and handling in C#?

Answer: Leaks occur when objects are kept alive unintentionally (events, static refs). Unsubscribe events, dispose IDisposable, avoid long-lived caches without eviction.

Cross-Questions: How to find leaks with dotMemory or PerfView?

QM3. Garbage collection in .NET?

Answer: Generational GC (Gen 0/1/2 + LOH) compacts and reclaims unreachable objects. Server vs workstation modes; background GC.

Cross-Questions: What triggers GC? How to reduce allocations?

QM4. Anonymous types?

Answer: Compiler-generated types with read-only properties created with 'new { }'; scope-limited.

Cross-Questions: Why can't you return anonymous types from public APIs?

QM5. StringBuilder usage?

Answer: Efficient for repeated string concatenations in loops; reduces intermediate allocations.

Example:

```
var sb = new StringBuilder();
for (int i=0;i<1000;i++) sb.Append(i);
var s = sb.ToString();
```

Cross-Questions: When is interpolation fine vs StringBuilder?

Section 9 – Coding Problems (Full C# Programs)

24. Count occurrences of each character in a string

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        Console.Write("Enter string: ");
        string input = Console.ReadLine() ?? string.Empty;
        var freq = new Dictionary<char, int>();

        foreach (char c in input)
        {
            if (freq.ContainsKey(c)) freq[c]++;
            else freq[c] = 1;
        }

        foreach (var kv in freq)
            Console.WriteLine($"{kv.Key}: {kv.Value}");
    }
}
```

25. Remove duplicate characters (preserve order)

```
using System;
using System.Collections.Generic;
using System.Text;

class Program
{
    static void Main()
    {
        Console.Write("Enter string: ");
        string input = Console.ReadLine() ?? string.Empty;
        var seen = new HashSet<char>();
        var sb = new StringBuilder();

        foreach (char c in input)
        {
            if (seen.Add(c))
                sb.Append(c);
        }
        Console.WriteLine(sb.ToString());
```

```
    }
}
```

26. Reverse each word in a sentence (preserve word order)

```
using System;
using System.Linq;

class Program
{
    static void Main()
    {
        Console.Write("Sentence: ");
        string sentence = Console.ReadLine() ?? string.Empty;
        string result = string.Join(" ",
            sentence.Split(' ', StringSplitOptions.RemoveEmptyEntries)
                    .Select(w => new string(w.Reverse().ToArray())));
        Console.WriteLine(result);
    }
}
```

27. First non-repeating character in a string

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        Console.Write("Enter string: ");
        string s = Console.ReadLine() ?? string.Empty;
        var freq = new Dictionary<char, int>();
        foreach (var c in s) freq[c] = freq.GetValueOrDefault(c) + 1;
        foreach (var c in s)
        {
            if (freq[c] == 1) { Console.WriteLine(c); return; }
        }
        Console.WriteLine("No non-repeating character.");
    }
}
```

28. Frequency of numbers in an integer array

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        int[] arr = { 1,2,2,3,1,4,2 };
        var map = new Dictionary<int,int>();
        foreach (int n in arr) map[n] = map.GetValueOrDefault(n) + 1;
        foreach (var kv in map) Console.WriteLine($"{kv.Key}: {kv.Value}");
    }
}
```

29. Missing numbers from a sorted array

```
using System;
```

```
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        int[] arr = {1,2,4,6,7};
        var missing = new List<int>();
        for (int i = arr[0]; i <= arr[^1]; i++)
        {
            if (Array.BinarySearch(arr, i) < 0)
                missing.Add(i);
        }
        Console.WriteLine(string.Join(", ", missing));
    }
}
```

30. Second highest number in an array (edge cases)

```
using System;

class Program
{
    static void Main()
    {
        int[] arr = {7,2,9,4,9,3};
        if (arr.Length < 2) { Console.WriteLine("Not enough elements."); return; }

        int first = int.MinValue, second = int.MinValue;
        foreach (int n in arr)
        {
            if (n > first)
            {
                second = first;
                first = n;
            }
            else if (n > second && n < first)
            {
                second = n;
            }
        }
        Console.WriteLine(second == int.MinValue ? "No second highest." : $"Second highest
    }
}
```

31. Sort and remove duplicates without built-ins

```
using System;

class Program
{
    static void Main()
    {
        int[] arr = {5,2,3,2,1,5};

        // Selection sort
        for (int i = 0; i < arr.Length - 1; i++)
        {
            int min = i;
            for (int j = i + 1; j < arr.Length; j++)
                if (arr[j] < arr[min]) min = j;
```

```
            (arr[i], arr[min]) = (arr[min], arr[i]);
        }

        // Print unique in order
        int? prev = null;
        for (int i = 0; i < arr.Length; i++)
        {
            if (prev == null || arr[i] != prev.Value)
            {
                Console.Write(arr[i] + " ");
                prev = arr[i];
            }
        }
    }
}
```

32. FizzBuzz variation (even/odd/prime)

```
using System;

class Program
{
    static bool IsPrime(int n)
    {
        if (n < 2) return false;
        for (int i = 2; i*i <= n; i++) if (n % i == 0) return false;
        return true;
    }

    static void Main()
    {
        for (int i = 1; i <= 100; i++)
        {
            if (IsPrime(i)) Console.WriteLine("FizzBuzz");
            else if (i % 2 == 0) Console.WriteLine("Fizz");
            else Console.WriteLine("Buzz");
        }
    }
}
```

33. Count Fizz, Buzz, FizzBuzz

```
using System;

class Program
{
    static bool IsPrime(int n)
    {
        if (n < 2) return false;
        for (int i = 2; i*i <= n; i++) if (n % i == 0) return false;
        return true;
    }

    static void Main()
    {
        int fizz=0, buzz=0, fizzbuzz=0;
        for (int i = 1; i <= 100; i++)
        {
            if (IsPrime(i)) { Console.WriteLine("FizzBuzz"); fizzbuzz++; }
            else if (i % 2 == 0) { Console.WriteLine("Fizz"); fizz++; }
            else { Console.WriteLine("Buzz"); buzz++; }
        }
```

```
        Console.WriteLine($"Fizz: {fizz}, Buzz: {buzz}, FizzBuzz: {fizzbuzz}");
    }
}
```

34. Get a specific number from array

```
using System;

class Program
{
    static void Main()
    {
        int[] arr = {1,4,6,8,10};
        Console.Write("Target: ");
        if (!int.TryParse(Console.ReadLine(), out int target))
        {
            Console.WriteLine("Invalid input.");
            return;
        }
        foreach (int n in arr)
            if (n == target) { Console.WriteLine("Found"); return; }
        Console.WriteLine("Not found");
    }
}
```

Section 10 – SQL Coding (Queries)

75. Top 5 customers by order count

```
SELECT TOP 5 CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
ORDER BY OrderCount DESC;
```

76. Second highest/lowest salary

```
-- Second highest
SELECT MAX(Salary) AS SecondHighest
FROM Employees
WHERE Salary < (SELECT MAX(Salary) FROM Employees);

-- With ties (rank)
SELECT Salary FROM (
  SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
  FROM Employees
) t WHERE rnk = 2;
```

77. Products with maximum order count

```
SELECT TOP 1 WITH TIES ProductID, COUNT(*) AS Cnt
FROM OrderLines
GROUP BY ProductID
ORDER BY Cnt DESC;
```

78. Product and category details (with pagination)

```
SELECT p.ProductID, p.Name AS Product, c.Name AS Category
FROM Products p
JOIN Categories c ON c.CategoryID = p.CategoryID
ORDER BY p.ProductID
OFFSET 0 ROWS FETCH NEXT 10 ROWS ONLY;
```

79. Latest N orders per customer

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate DESC) AS rn
  FROM Orders o
) t WHERE rn <= @N;
```

80. Customers who purchased in both 2023 and 2024

```
SELECT CustomerID
FROM Orders
WHERE YEAR(OrderDate) IN (2023, 2024)
GROUP BY CustomerID
HAVING COUNT(DISTINCT YEAR(OrderDate)) = 2;
```

81. Customers who ordered a specific product > 3 times

```
SELECT CustomerID
FROM OrderLines
WHERE ProductID = @ProductId
GROUP BY CustomerID
HAVING COUNT(*) > 3;
```

82. Top best-selling products with total sales amount

```
SELECT TOP 10 ol.ProductID, SUM(ol.Quantity * ol.UnitPrice) AS TotalSales
FROM OrderLines ol
GROUP BY ol.ProductID
ORDER BY TotalSales DESC;
```

83. Max orders count + category/product name

```
WITH C AS (
  SELECT p.ProductID, p.Name AS ProductName, c.Name AS CategoryName,
         COUNT(*) AS OrderCount
  FROM OrderLines ol
  JOIN Products p ON p.ProductID = ol.ProductID
  JOIN Categories c ON c.CategoryID = p.CategoryID
  GROUP BY p.ProductID, p.Name, c.Name
)
SELECT TOP 1 WITH TIES * FROM C ORDER BY OrderCount DESC;
```